

Backlink Admissibility in Cyclic Proofs for the Calculus of Inductive Constructions

Gavin Mendel-Gleason

Scidonia Limited
gavin@scidonia.ai

Abstract. Cyclic proofs close goals by backlinking to previously-seen configurations, discovering induction principles post-hoc rather than prescribing them upfront. We prove that backlinks are *admissible* in the Calculus of Inductive Constructions (CIC): every valid cyclic proof, presented as a finite directed graph satisfying a budget-based progress condition, can be residualised to a standard CIC term using only the native fixpoint binder ($\text{fix}(A).t$) for recursion, and the residual is observationally indistinguishable from the root goal. Crucially, we work throughout with *finitely presented* proof structures; this finiteness is what allows the soundness argument to proceed by well-founded induction entirely within CIC’s inductive fragment, with no appeal to coinduction or infinite proof trees. The proof is mechanised in Rocq and relies only on *functional extensionality*—a standard, widely-accepted axiom for CIC—with no other axioms or uses of **Admitted**.

1 Introduction

Cyclic proof theory [1,2] extends standard proof systems with backlink rules that allow a premise to reference a previous goal, forming a directed cycle. The cycle is sound if it satisfies a global *trace condition*: every cycle must contain a *progress event*—a case-split on a structurally smaller term. Brotherston and Simpson prove that for first-order logic with inductive definitions, backlinks are *admissible*: every cyclic proof corresponds to a standard proof with an explicit induction rule.

This paper extends their result to the Calculus of Inductive Constructions (CIC) [3,5], the type theory underlying Rocq. We prove that every rooted, ranked cyclic proof can be residualised to a standard CIC term using only $\text{fix}(A).t$ for recursion, and that the residual is observationally equivalent to the root goal.

Finite presentations and the inductive fragment. A fundamental point separates our account from Brotherston’s original treatment [1]. Brotherston works with the *infinite regular tree* obtained by unrolling a cyclic proof, and proves soundness by reasoning about infinite branches of that tree—an argument that inherently requires some form of limit reasoning (König’s lemma, or a semantic argument about infinite derivations).

We work throughout with the *finitely presented* proof structure itself. A cyclic proof is a *finite directed graph* whose vertex set V is finite, decidable, and countable (concretely, $\mathbb{N}$), with each vertex labelled by a typing judgement and its outgoing edges labelled by the rule premises applied at that vertex. This finiteness is not a restriction in practice: every cyclic proof produced by a proof-search or supercompilation procedure is finite by construction.

The finiteness of V is what makes the soundness argument purely inductive. The progress condition guarantees a strictly decreasing lexicographic measure across every back-edge traversal in the *finite* graph—so the well-founded recursion terminates. No unrolling to an infinite tree is needed; no coinductive reasoning is required. The proof term produced by the `backlink_admissible` theorem in Rocq is a fully normalising, purely inductive term that Rocq’s kernel verifies without any `cofix` or classical axioms.

In short: *the finite presentation of a cyclic proof induces an inductive proof term*. This is the mechanism by which cyclic proofs gain their expressive power while remaining within CIC.

Future work: coinductive generalisation. An interesting direction is to extend this approach to proof systems where the cyclic structure is not given in advance as a finite graph, but is instead generated lazily or represented as an infinite but regular tree—as arises naturally in proof-search over infinitely-branching or higher-order systems. In such a setting the appropriate framework is *coinductive*: the progress condition would need to be reformulated as a coinductive invariant on the infinite unrolling, and soundness would become a productivity argument in the spirit of Coquand’s coinductive proof theory or the global trace conditions of Brotherston–Simpson. We leave this generalisation as future work.

2 Term calculus and typing

Terms.

$$t, A ::= x \mid \star_i \mid \Pi x:A. B \mid \lambda x:A. t \mid t u \mid \mu x:A. t \mid I \mathbf{a} \mid c \mathbf{a} \mid [t \mid \mathbf{b}]_C^I$$

where x is a de Bruijn index, $\star_i$ a universe, $\mu x:A. t$ a fixpoint (x the recursive call), I an inductive type, c a constructor, and $[t \mid \mathbf{b}]_C^I$ elimination with motive C . Arrow-annotated letters $\mathbf{a}, \mathbf{b}$ denote (possibly empty) vectors of terms. Σ records inductive signatures.

Contexts (telescopes). A context Γ is a *dependent telescope*: a list of types where each entry may depend on all earlier variables. Formally $\Gamma ::= \cdot \mid \Gamma, A$ with shift-aware de Bruijn lookup:

$$\frac{}{\cdot \text{ ctx} \quad (\text{CTXNIL})} \quad \frac{\Gamma \text{ ctx} \quad \Gamma \vdash A : \star_i}{\Gamma, A \text{ ctx} \quad (\text{CTXCONS})}$$

Variable x in $\Gamma = A_n, \dots, A_0$ has type $\Gamma(x) = A_x \uparrow^{x+1}$, i.e. A_x shifted to account for the $x+1$ binders between A_x and the use site. In the mechanisation `ctx` is list `T.tm` with `ctx_lookup` applying `T.shift 1 0` at each recursive step.

Substitution formation. The judgement $\Delta \vdash \sigma : \Gamma$ (written `has_subst` in the mechanisation) says that the list $\sigma = [u_0, \dots, u_{n-1}]$ is a well-typed simultaneous substitution from Γ into Δ :

$$\frac{}{\Delta \vdash [] : \cdot} \text{ (SUBNIL)} \quad \frac{\Delta \vdash \sigma : \Gamma \quad \Delta \vdash u : \sigma(A \uparrow)}{\Delta \vdash u :: \sigma : \Gamma, A} \text{ (SUBCONS)}$$

where $\sigma(A \uparrow) = \text{subst_list } \sigma (A \uparrow)$ substitutes the already-accumulated σ into A (shifted once). Applying σ to a term t under Γ yields a well-typed term under Δ (the substitution lemma, `has_type_subst`).

Typing. $\Sigma; \Gamma \vdash t : A$ is standard for CIC [3,5]; Σ is implicit below, $\uparrow$ denotes shift.

$$\begin{array}{c} \frac{}{\Gamma \vdash x : A} \text{ (VAR)} \quad (\Gamma(x)=A) \qquad \frac{}{\Gamma \vdash \star_i : \star_{i+1}} \text{ (SORT)} \\[10pt] \frac{A : \star_i \quad A :: \Gamma \vdash B : \star_j}{\Gamma \vdash \Pi x:A.B : \star_{\max(i,j)}} \text{ (PI)} \qquad \frac{A : \star_i \quad A :: \Gamma \vdash t : B}{\Gamma \vdash \lambda x:A.t : \Pi x:A.B} \text{ (L)} \\[10pt] \frac{t : \Pi x:A.B \quad u : A}{\Gamma \vdash t u : B[u/x]} \text{ (@)} \qquad \frac{A : \star_i \quad A :: \Gamma \vdash t : A \uparrow}{\Gamma \vdash \mu x:A.t : A} \text{ (MU)} \\[10pt] \frac{\Sigma(I) = \Sigma_I}{\Gamma \vdash I : \star_{\ell(\Sigma_I)+1}} \text{ (IND)} \qquad \frac{\Sigma(I) = \Sigma_I \quad \Sigma_I(c) = \tau_c \quad a_i : \tau_{c,i} \forall i \quad r_j : I \forall j}{\Gamma \vdash c \mathbf{a} : I} \text{ (ROLL)} \\[10pt] \frac{t : I \delta \quad x:I \delta, \Gamma \vdash C : \star_i \quad b_c : \Pi \tau_c. C[c \mathbf{y}/x] \forall c}{\Gamma \vdash [t \mid \mathbf{b}]_C^I : C[t/x]} \text{ (CASE)} \end{array}$$

Reduction. CBN weak-head: $(\lambda x:A.t) u \rightarrow t[u/x]$, $\mu x:A.t \rightarrow t[\mu x:A.t/x]$, $[c \mathbf{a} \mid \mathbf{b}]_C^I \rightarrow b_c \mathbf{a}$, plus congruence under application and case scrutinee. $\rightarrow^*$ is the closure; $t \Downarrow v$ means $t \rightarrow^* v$ with v a value.

3 Cyclic sequent calculus

Finite configuration graphs. A *configuration* is a typing judgement $\Gamma \vdash t : A$. A *configuration graph* b is a finite structure with vertex set V (implicitly $\{0, \dots, \text{next} - 1\}$), a labelling $\ell(b, \cdot) : V \rightarrow \mathbf{Config}$, a successor map $\sigma(b, \cdot) : V \rightarrow \text{list}(V)$, and a distinguished root $v_0 \in V$. In the mechanisation the graph is a `cfg_builder` record with finite `gmap nat` tables; well-formedness (`cfg_builder_well_formed`) ensures that every successor is itself a labelled vertex.

A graph is a *rooted pre-proof* if every $v \in V$ satisfies one of the local rules below (read bottom-up):

Asynchronous rules (deterministic). Applied eagerly; each reduces the goal to a single premise.

$$\frac{\Gamma \vdash t[u/x] : A}{\Gamma \vdash (\lambda x:A. t) u : A} (\beta) \qquad \frac{\Gamma \vdash t[\mu x:A. t/x] : A}{\Gamma \vdash \mu x:A. t : A} (\mu)$$

$$\frac{\Gamma \vdash b_c \mathbf{a} : A}{\Gamma \vdash [c \mathbf{a} \mid \mathbf{b}]_C^I : A} (\iota)$$

Synchronous rule (choice point). A case-split on a neutral scrutinee x creates one premise per constructor c of I , each substituting the constructor pattern into the c -th branch:

$$\frac{\Gamma, \mathbf{y}_0 \vdash b_0[0 \mathbf{y}_0/x] : A \quad \cdots \quad \Gamma, \mathbf{y}_n \vdash b_n[n \mathbf{y}_n/x] : A}{\Gamma \vdash [x \mid \mathbf{b}]_C^I : A} (\text{split})$$

A SPLIT vertex is a *progress vertex*: it records a case-split on a neutral scrutinee and is the locus of structural decrease that underpins the trace condition.

Cyclic backlink (closure). A vertex may be closed by a *back-edge* to a previously-seen configuration under a substitution σ :

$$\frac{\Delta \vdash \sigma :_s \Gamma}{\Delta \vdash t : A} (\text{back})$$

where there exists $v' \in V$ with $\ell(b, v') = \Gamma \vdash t_0 : A$ and $t = t_0[\sigma]$. The substitution σ is *not* the result of further supercompilation: it is produced by anti-unifying the current configuration with v' 's configuration, extracting the instantiation terms that fill the holes of the generalised pattern.

Global progress condition. A rooted pre-proof satisfies the progress condition if every directed cycle contains at least one progress vertex. In the mechanisation this is checked via a *budget trace*:

Definition 1 (Budget trace graph). *Given a configuration graph G with progress predicate `prog` and initial budget $B = |V|$, the budget trace graph has vertices $V \times \{0, \dots, B\}$ and an edge $(v_1, k_1) \rightarrow (v_2, k_2)$ whenever $v_1 \rightarrow v_2$ is an edge in G and:*

- v_1 is a progress vertex and $k_2 = k_1 - 1$, or
- v_1 is not a progress vertex and $k_2 = k_1$.

The progress condition holds—written $\mathcal{C}(b, v_0)$ —iff the budget trace graph is acyclic. In code: `trace_condition_ok(b)  $\equiv$   $\neg$ has_nonprogress_cycle(b)`. Since the budget starts at $B = |V|$ and can decrease at most $|V|$ times, acyclicity of the budget trace follows from the progress condition (`trace_rank_monotone`, `trace_rank_strict_on_progress`).

4 Residualisation

The backlink rule appears only in the proof graph. A *residualisation* function $\mathcal{R}(b, v, d, \rho)$ reads back the graph into a CIC term by traversing the successors of v , replacing backlinks with μ binders and recording previously-seen vertices in a *fix environment* $\rho : \mathbb{N} \rightarrow \mathbb{N}$ (graph vertices to de Bruijn depths). $\mathcal{R}$ is defined by well-founded recursion on the budget counter.

- **Leaf**: returns t where $\ell(b, v) = \Gamma \vdash t : A$.
- **Asynchronous** (single successor w): $\mathcal{R}(b, v, d, \rho) = D(\mathcal{R}(b, w, d, \rho))$ where D re-abstracts the driving step (β , μ -unfold, ι).
- **Split** (multiple successors): each branch residualised independently, combined into $[\cdot \mid \mathbf{b}]_C^I$.
- **Backlink** ($v \notin \text{dom}(\rho)$): inserts $\mu x:A. t$ where t residualises the source with x as self-reference.
- **Already seen** ($v \in \text{dom}(\rho)$): returns the corresponding de Bruijn variable.

The result uses only λ , application, μ , $[\cdot \mid \mathbf{b}]_C^I$, and $c\mathbf{a}$ — no backlink constructs remain.

Why finiteness is essential. The residualisation terminates because the budget counter strictly decreases on every back-edge and is bounded by $|V|$ which is finite. If V were infinite, the recursion would not terminate and the function could not be given a type in CIC without coinduction. The finite vertex set is thus the direct reason the residualiser produces an *inductive* proof term.

5 CIU equivalence

The soundness guarantee is *CIU equivalence*: for closed terms,

$$t \approx_{\text{ciu}} u \equiv (\forall \sigma v. t[\sigma] \Downarrow v \Rightarrow u[\sigma] \Downarrow v) \wedge (\forall \sigma v. u[\sigma] \Downarrow v \Rightarrow t[\sigma] \Downarrow v)$$

where $\sigma : \mathbb{N} \rightarrow \text{tm}$ is an arbitrary de Bruijn substitution. This is the open CIU equivalence lifted to all closing substitutions.

Each local rule preserves CIU equivalence. The asynchronous rules are single reduction steps (CIU-preserving by `step_ciu`); the split rule applies CIU-congruence for neutral scrutinees; the back rule becomes a μ binder whose unfolding reproduces the source derivation.

Theorem 1 (CIU soundness — general). *Let b be any closed configuration graph satisfying the trace condition (`builder_succ_closed(b) ∧ trace_condition_ok(b)`), with $\ell(b, v) = \Gamma \vdash t : A$. Then for any depth d and fix environment ρ with $v \notin \text{dom}(\rho)$:*

$$t^{\uparrow d} \approx_{\text{ciu}} \mathcal{R}(b, v, d, \rho).$$

Proof (Proof sketch). By well-founded induction on the budget counter (lemma `residualise_cfg_ciu`). At each vertex, each local rule is CIU-preserving: asynchronous rules are single reduction steps; the split rule uses CIU-congruence for neutral scrutinees; a backlink inserts a μ binder whose unfolding reproduces the source derivation. Well-foundedness uses `lex_lt_wf`: every progress edge strictly decreases the budget (`trace_rank_strict_on_progress`), and the budget is bounded by $|V|$. There is no coinductive step, no appeal to König’s lemma, and no reasoning about infinite unrollings.

The hypothesis requires only that the graph is *closed* and *passes the trace check* — it does not matter how the graph was produced. Any tool that generates such a finite graph (supercompiler, proof-search procedure, hand-written cyclic proof) yields the same guarantee.

6 Admissibility

Theorem 2 (Backlink admissibility). *Let b be any closed configuration graph satisfying the trace condition, with root vertex v labelled $\Gamma \vdash t : A$. Then there exists a standard CIC term t_{std} such that*

$$t \approx_{\text{ciu}} t_{\text{std}} \quad \text{and} \quad t_{\text{std}} = \mathcal{R}(b, v, 0, \emptyset).$$

Proof. Take $t_{\text{std}} = \mathcal{R}(b, v, 0, \emptyset)$. By construction, t_{std} uses only standard CIC constructors (λ , application, μ , $[\cdot \mid \mathbf{b}]_C^I$, $c \mathbf{a}$). CIU equivalence is the $d=0$, $\rho=\emptyset$ case of Theorem 1.

The proof term is purely inductive (no `cofix`, no `Admitted`). The sole non-constructive ingredient is *functional extensionality* ($\forall f g, (\forall x, f(x) = g(x)) \Rightarrow f = g$), which enters via Autosubst’s substitution lemmas and our typing substitution proofs. This axiom is standard and widely accepted in CIC mechanisations; it does not affect the computational content of the proof term.

7 Discussion

Finite presentation is essential, not incidental. The admissibility result requires three things of a proof graph: (i) it is finite ($|V| < \omega$); (ii) it is closed (every successor is a labelled vertex, i.e. `builder_succ_closed`); and (iii) it satisfies the trace condition (every cycle contains a progress vertex, i.e. `trace_condition_ok`). All three are needed. In particular, finiteness bounds the budget $B = |V|$, makes the budget trace graph finite, and enables well-founded recursion.

Relationship to Brotherston–Simpson. Brotherston and Simpson prove admissibility for first-order logic by unrolling cyclic proofs to infinite regular trees and applying a semantic argument about infinite derivations. Our proof is structurally simpler: we never leave the finite graph. Instead of reasoning about infinite

branches, we reason about the budget counter, which decreases on every back-edge and is bounded by $|V|$. The finiteness of V is what allows admissibility to be expressed as `well_founded_induction` rather than as an ordinal assignment or compactness argument.

The inductive/coinductive boundary. In CIC there is a sharp boundary between *inductive* types (well-founded recursion, terminating) and *coinductive* types (guarded corecursion, productive). Cyclic proofs appear at first to belong on the coinductive side: they contain cycles, and their natural models are infinite regular trees. The central insight of this paper is that when the cyclic proof is *finitely presented*—i.e. given as a concrete finite graph—the progress condition provides enough structure to carry out the soundness argument entirely on the inductive side. The finite graph, together with its progress condition, induces a well-founded measure, which induces a terminating recursion, which induces a normalising proof term.

Acknowledgements. Thanks to James Brotherston for the question that motivated this paper.

References

1. Brotherston, J.: Cyclic proofs for first-order logic with inductive definitions. In: TABLEAUX (2006)
2. Brotherston, J., Simpson, A.: Sequent calculi for induction and infinite descent. In: LICS (2011)
3. Coquand, T., Huet, G.: The calculus of constructions. *Information and Computation* **76**(2–3), 95–120 (1988)
4. Mendel-Gleason, G.: Supercompilation corresponds to cyclic proof in the calculus of inductive constructions (2025), under review
5. Paulin-Mohring, C.: Inductive definitions in the system Coq: Rules and properties. In: *Typed Lambda Calculi and Applications*. pp. 328–345. Springer (1993)